

MuSR 部分实验报告 (论文 §5 全景)

2026-07-21

摘要

本报告给出 SAVI 在 belief-tracking 任务上的实验结果。核心主张是: 在没有免费验证器的真实任务上, SAVI 能修复“采样 — 投票”范式修不动的失败, 且增益来自解码方式本身, 不依赖额外信息。实验在信念追踪基准 MuSR object_placements(Qwen3-4B) 上分三步完成。(1) 搭建测试台: 四级认证漏斗从 256 题中认证出 19 道投票修不动的题 (另有 3 道知识对照); 流行率测量说明同型失败步 (全票写错, point mass) 在未筛选题目上并不罕见, 测试集严苛但不是孤例。(2) 排除池内替代解释: learned-verifier 重排与 best-of-N 逐题相等, gold-facts 掩码给出的池内上界也只有 3/19。“在采样池里挑得更聪明”复现不了修复, 缺的候选不在池里; SAVI 的 support 轴 (枚举候选) 补的正是这一块。(3) 验证 SAVI 并归因: SAVI 实例化为枚举可达信念状态、每步锚一句模型自写的事件句、似然打分、Viterbi 取全局最优路径, 零人工标注修复 9/19, 知识对照 0/3。消融把增益定位到“每步一句具体、成句断言”的存在 (4→9; 内容改对仅 +1); 机制探针显示似然打分特异、频率打分非特异, 说明 SAVI 起效走的是“认得出”(recognize) 通道。失败本身是分叉处的自信写错: 给足信息后仍有 80% 写错, 不是缺信息。这也解释了修复为什么要由解码侧完成。

1 实验目的

- 存在性:** 说明 commitment failure(承诺失败) 在真实基准上存在。认证出一个 can't 集: 失败属于“知道但用不对”, 而非知识缺失。(对应认证漏斗, 第三节)
- 转化:** 检验全局解码 (枚举信念状态 + 模型自身似然打分 + 精确 Viterbi) 能否让模型在这批题上实现 can't → can。(对应第四、五节主结果: 9/19 vs 投票 0/19)
- 归因与定位:** 把转化的功劳归到具体成分 (支撑 / 打分 / 每步锚句), 并定位失败机制 (分叉处自信地错)。(对应四个 arm 消融 + 分叉重放, 第五节)

2 Benchmark 及测试用例介绍

MuSR(Sprague et al. 2024) 是一个多步软推理基准, 用 GPT-4 生成长叙事、人工校验答案, 有三个子任务。本实验用 object_placements 子任务, 它是真实的信念追踪 (belief tracking) 基准: 长叙事里物体被移动多次, 每次移动只有部分角色看见; 问“角色 X 认为物体 Y 在哪 / 会去哪找”, 多选题。

实际的例子:

叙事: Ann 把钥匙放进抽屉后出门; 她不在时 Ben 挪到盒子。

问题: Ann 会去哪找钥匙? 选项: 抽屉 / 盒子

正确答案是“抽屉”(她没看见挪动)——但模型顺着最近提到的「盒子」写下去, 就回不了头。

选择该任务的三个结构性原因:

- 状态可枚举:** 每一步维护一张信念状态表 —— (观察者, {物品: 位置}), 即每个观察者对每个物品位置的信念。表的取值有限, 每步可达的候选状态能穷举出来。
- 答案可直接读出:** 终端信念表查 (X, Y) 一格就是答案, 弃用模型自写的“ANSWER: ...”行, 以排除“推到了正确终端状态、却把最后的答案行写错”这层混淆 (读出消融单独验证过只 +1)。
- 和 Countdown 的关键区别:** 域内没有免费的可执行检查器。“免费”指同时满足三条: 不需要训练、不需要真实事实/人工标注、从任务规则机械可算。Countdown 的检查器从域规则免费推出 (执行算术求解), MuSR 的检查只能对照真实事实, 三条至少违反一条。因此公平的强基线取多数投票。

主要实验模型与采样协议: 生成器 Qwen3-4B; 采样 T=1.0、top_p=1.0、N 最大 1024、

max_new_tokens=2048。
主基线 = self-consistency(多数投票):N 条完整链各解析最后一行 ANSWER: k, 取答案众数。

3 实验设置: 认证漏斗 (如何选题) 与基线定义

后续主张都建立在“这 19 题确实修不动”上。总原则: 每道门剔除一类并非真正修不动的题, 对应如下:

论文	内部阶段	操作	结果	排除的替代解释
起点	—	object_placements 256 题	—	—
F1	递增预算	错误众数题从 N=32 升到 N=1024	107 → 105, 近乎持平	“预算不够”; 预闭合后面所有比较的 token-parity 问题
F2	四门认证	对 105 道存活题过四道门	26 道认证	见下
F3	自提取控制	让模型自己提取事实再作答 (两种注入方式)	只修 5 道, 残余 21	“好 prompting 就够了”
F4	模板预检	结构化 prompt 下重跑投票	19/21 仍错, 最终 19	“结构化模板本身就能修”

F1(budget escalation,256 → 105)

- 判据: 多数投票的众数答案错 (投票层面, 不是单条链错), 且预算从 N=32 加深到 N=1024 后众数仍错。结果 107@N=32 → 105@N=1024, 近乎持平。
- 隐含约束:“不是预算不够”。投票加 32 倍 token 都不改答案, 后续的比较不会再被“预算不足”这一解释推翻。

F2(four checks,105 → 26)

1. 统计稳定 (排运气, 剔 4): 对“正确答案占比”与“错误众数占比”各计算 CI(N=1024), 要求两区间分离 (正确率上界 < 错误众数下界) 才算真做不对。
2. 给定 gold-fact(排知识失败, 剔 46): 在 prompt 前注入观察日志 (按时间逐条“谁把什么移到哪、谁看见了/没看见”), 剔除注入后仍做不对的题目; 这部分题目后续用作 knowledge control。
3. 不同规模过滤 (排规模, 剔 4):Qwen2.5-Instruct {0.5, 1.5, 3, 7}B; 只要任一规模的模型“稳定解开”即剔除。
4. 鲁棒性检验 (排种子巧合, 剔 0): 换第二种子, 要求同一个错误选项仍为众数。

F3(self-extraction control,26 → 21)

- 操作: 让模型先自己从叙事提取事实、再作答 (两种注入方式)。只修好 5 道“易题”(其余题中位采样正确率 0.044)。
- 记账语义:F3/F4 修好的 5+2 道记为被剔除, 不进任何方法成绩; 能被好 prompting 修好的题不属于 can’t 集。

F4(template pre-check,21 → 19)

- 操作: 在与解码侧共享的结构化模板 (BELIEF[t] 信念表逐事件格式 + ANSWER 行) 下重跑多数投票 (N=64)。
- 含义:can’t 不是格式伪影, 投票基线与解码 prompt-matched: 双方共享“信念表格式 + 任务框架”, 解码独有的只剩单步条件化、事件句、机械读出。

最终认证集构成:19 题 (+3 知识对照)= 15 道有覆盖 (采样到了、投票投错)+ 4 道零覆盖 (1024 次采样一次都没有对)。

知识对照 (knowledge controls): 从 F2 (ii) 剔除的 46 道知识失败题 (喂 gold-fact 仍错) 里按种子抽样得到, 用作“不该被修”的反向检验: 任何方法把它们“修好”, 就说明该修复对承诺失败不特异 (可能泄露了答案), 其修复数作废。

流行率分析 (prevalence, 论文 §5.2)

认证漏斗回答“哪些题修不动”，流行率回答“这种失败是不是罕见现象”。定义**全票写错步** (unanimous-wrong): 该步所有采样链写出同一个错误后继，经验后继分布是 point mass(概率全压在一个点、零分歧)。结果: 可判定题 (共 40 道) 中 $32/40 = 0.800$ 至少含一个全票写错步 (CI [0.675, 0.925]); 可判定层 $67/115 = 0.583$ 本身全票写错。

与认证集选取的关系: 未筛选切片上，八成可判定题都带有与认证集同型的失败步 (全票写错，主形态是照抄不更新)，只是多数失败步没落在决定答案的位置。认证的 19 题就是失败恰好落在决定性分叉上、绕不过去的那部分题，不是筛选制造出来的罕见标本。

4 实验组与对照组的设置

4.1 对照系: 池内聚合 (论文 §5.3)

问题: 同一批采样链，聚合得更聪明够不够? 两个升级 arm 都基于结构化 prompt 的模型输出，且候选全部来自采样池，两个 arm 都不提供、不生成任何新候选状态:

- **learned-verifier:** 训练一个步级事实一致性探针。用法: 主 arm 是信念 trellis 上的 λ 软加权解码; 同一 verifier 另设 BoN 消融 arm(逐链取步级分均值、argmax 选链)。实测两者逐题完全相等 (净差 0, CI[0,0]), 即该 arm 等价于“逐步打分、选总分最高的链”;
- **gold-facts mask[ORACLE]:** 完全离线 (post-hoc): 先把 N 条链全部采完，解析每层 BELIEF 行 $\rightarrow$ 规范化 $\rightarrow$ 跨链归并成 trellis(归并率中位 0.963) $\rightarrow$ 用 gold-fact 一致性掩码禁掉矛盾结点 $\rightarrow$ 在剩余 trellis 上解码最优路径。机制上允许跨链拼接，实测拼接占比 0，修复全在单链内。

本层结论: learned-verifier = BoN(trellis 结构零增量); 跨链拼接机制可用但没有被用上，修复只发生在完整采样链内; 即使给 gold-fact 信号，池内上界也只有 3/19。问题不在挑选方式: 缺的候选根本不在池子里。

池内聚合方式	fixed / 19	备注
majority voting	0	主基线
learned-verifier rerank(= best-of-N)	净 +2, CI [0, 5]	与 0 不可分; 逐题与 BoN 相等
gold-facts mask (oracle), sampled chains only	3	全在已有链内; 跨链拼接 0

表 1 对照系 (池内聚合) 结果

4.2 实验系: 枚举解码 $\times$ 事件句四个 arm(论文 §5.4)

每步打分的模板 (确定性 teacher forcing, 无采样; 对每个 (层, 前状态, 候选) 三元组各算一次):

```

prefix:
[叙事全文] + [问题 + 选项]
[t-1 的信念状态] BELIEF[t-1]: {"Ben": {"keys": "drawer"}, ...}
[t event] Event 3: the keys are moved to the box.
Write exactly one line recording what every character now believes ... in the same format.
+
target:
[枚举可达状态] BELIEF[t]: {"Ben": {"keys": "box"}, ...}  $\leftarrow$  打分只量这一段

```

唯一被操纵的变量: 模板里的 [event] 事件句, “the keys are moved to the box.”, 作为 anchor 锚定当前事件步:

arm	事件句内容
none	整行删掉 (只留一个空行)
placeholder	固定占位句, 语法完好、只透传步号 t, 零事实内容
model-written	模型自己从叙事里提取 (解析错了不剔除)
gold	真实事实注释

打分与归一化: 对 target 行逐 token log-softmax 求和得原始分; 主归一化 = 同一 (层, 前状态) 候选集内的 log-softmax, 即每个转移归一成候选集内的条件对数概率; 路径分 = 逐层边权和, 精确 Viterbi 取全局 argmax。

对照: 除 3 道知识对照外, 另设频率打分对照: 同一枚举支撑上把似然换成采样频率打分, 供 5.2 的 writes vs recognizes 区分使用。

5 实验结果与分析

5.1 结果与分析

	fixed / 19	+ gold mask (ceiling)
majority voting over the sampled pool	0	—
learned-verifier rerank of the pool(= best-of-N)	净 +2, CI [0, 5]	3
none	3	5
placeholder (no content)	4	6
model-written (no human-labeled input)	9	12
gold	10	15

表 2 主结果 (各方法修复数 /19 与 gold-fact 掩码上界)

- **机制:** 具体断言让每步的后继分布坍塌; 对全路径解码来说, “分布坍塌了” 比 “坍塌到哪个后继” 更重要。
- **这组梯度怎么读:** 只有格式 (占位句) 几乎不起作用 (none $\rightarrow$ placeholder 仅 3 $\rightarrow$ 4); 主要增益在 “每步一句具体的、成句的断言”(placeholder $\rightarrow$ model-written, 4 $\rightarrow$ 9); 断言内容改对只再值约一题 (model-written $\rightarrow$ gold, 9 $\rightarrow$ 10)。
- **自写句子大多有错, 修复照样发生:** 19 题没有一道全部提取正确, 中位每题 3 个内容槽只对 1 个; event 内容质量与修复的相关弱 (Spearman 0.31), 有 6 道修复完全来自不精确的提取。
- **仅枚举本身不起作用:** 不依赖 event 句时, 枚举几乎没有修复效果; gold arm 修复 4 道零覆盖题中的 3 道, 删去 event 句后修复为 0。
- **特异性:** 似然打分下四个 arm 的知识对照都是 0/3。

5.2 机制定位

- **失败不是缺信息:** 分叉重放中, 给足正确历史和 gold-fact 句, 认证集分叉处的众数后继只有 20% 命中 gold-fact, 知识对照为 67%。模型是在分叉处自信地写错; 认证阶段所说的 “承诺失败” 由此被定位到分叉节点。
- **writes vs recognizes:** 似然打分 (recognize) 特异 (知识对照 0/3), 频率打分 (write) 非特异 (修 2/3 知识对照), 起作用的是 recognize 一侧。另外, 模型预测似然与实际采样频率的排序相关性 median $\rho=0.400$: TF 似然对模型实际采样频率的排序保真度不高, 不能解读为模型的 “信念”。

6 思考与下一步计划

1. **从信息角度分析增益** (论文 §6 “Same information, a different read-out”): SAVI 计算的每一个量都是采样器所读同一上下文的确定性函数。方法没有引入任何关于答案的新信息, 增益只能落在提取泛函上:
 - **support:** 采样链只在它恰好写出的那一个后继上花费模型的判断; 枚举把模型的条件判断读到每个可达信念状态上, 包括它自己永远不会写出的反事实状态。仅有 trellis 结构没有帮助, 但它是下游设计的必要条件;
 - **scoring:** 采样用模型写出的东西投票, 而认证题恰恰是模型在分叉处自信写错的地方; 似然打分改读 “认得出”, 并被每步锚句锐化 (4 $\rightarrow$ 9);
 - **decision:** 采样器每步承诺、不可逆, 给出确定答案; 解码器全局只承诺一次, 过程中不锁死某个判断,

答案从终端信念表机械读出 (读出单独 +1)。

增益不来自信息量, 而来自利用率: 采样方法每条链的每一步都只把判断花在抽中的那一个后继上, SAVI 在每一步考虑全部可达后继, 对同一信息的利用率更高。

2. **E0(aliasing audit 前置)**: E0 要求 BAR 定义在 beam baseline 的生成文本上。本实验最接近的量是 4.1 里 gold-facts mask 构建 trellis 时测得的跨链归并率中位 0.963: 信号足够强 (有大量内容可合并), 但它来自 $T=1.0$ 采样 + 结构化 BELIEF 模板, 与 E0 的 beam 设定不能直接等价, 需要扩展采样条件。
3. **BAR 预测搜索收益 (Aliasing rate as a predictor of search benefit)**: 在 4.1 的对照实验中, 归并率高达 0.963, 但 pool 内使用 SAVI 的 trellis 方式收益为零 (与 BoN 逐题等价)。原因是 BAR 只测量“写出来的内容有多冗余”, 观察不到“从没被写出来的状态”; 决定性分叉上 pool 是 point mass, 正确选项不在池里, 归并得再好也选不出。因此要用 BAR 判断 SAVI 何时起作用, 至少要补一个衡量决定性层覆盖 / 多样性的变量 (point mass 检查, 流行率的做法可迁移)。
4. **KV cache sharing**: 目前实验中的归并率可以直接给 K'/K 提供 real-problem 数据点: 0.963 意味着 state-level 实际束宽 K' 远小于 K , 内存节省潜力大。这是纯粹的效率主张, 与“trellis 对准确率零增量”的结论不冲突。
5. **上下文容量与推理能力**: 模型的上下文容量和推理能力远不成比例: 上下文能容纳很多步推理, 但往往在步数还不大时出错频率就已经很高, 且后续出错与带错的历史文本相关。与 MuSR 的关联:
 - **符合前提**: 认证题的失败发生在远离上下文上限的位置 (叙事 + 链远小于 8192 token 上限, 而 80% 可判定题仍含全票错步)。出错不是“存不下”, 是“用不好”, 与“推理步数远小于容量时错误已高发”的观察一致;
 - **状态化去噪**: 走识别通道 (枚举 + 似然), 比较 (规范化重置) 与 (不重置) 两个对照的结果差异。
6. **拓展实验**: 在更多 benchmark (ZebraLogic 逻辑谜题、seqbench 迷宫问题等) 上验证 SAVI 的效果。
7. **模型规模限制**: 目前的模型规模 ($\leq 7B$) 上可能不存在“belief”这一概念, 计划验证 scale up 之后的效果。
8. **训一个可迁移的步级 verifier**: 目前的 verifier 是任务特定的, 在新任务上需要重新设计, 迁移性差; 考虑结合 LLM 入手。